



Multi-layer Perceptrons

Universal Approximation • Forward Propagation • Backpropagation

What is a Multi-layer Perceptron?



Definition

A feedforward artificial neural network with multiple layers

Consists of an input layer, one or more hidden layers, and an output layer

Each layer is fully connected to the next layer

Also called Deep Feedforward Networks or Deep Neural Networks

Key Differences from Single-layer Perceptron

- Can solve non-linearly separable problems (like XOR)
- Multiple layers enable hierarchical feature learning
- Requires more sophisticated training (backpropagation)

MLP Architecture

Network Layers

Input Layer: Receives raw features
Number of nodes = feature dimensions

Hidden Layer(s): Process and transform features
Apply non-linear activations
Can have multiple hidden layers

Output Layer: Produces final predictions
Number of nodes = output classes

Mathematical Notation

L = total number of layers

n^l = number of neurons in layer l

W^l = weight matrix for layer l

b^l = bias vector for layer l

a^l = activation output from layer l

z^l = weighted sum at layer l

Example: 3-layer network

- Input: 784 nodes (28×28 image)
- Hidden: 128 nodes with ReLU
- Output: 10 nodes (digit classes)

Universal Approximation Theorem

Theorem Statement

A feedforward neural network with a single hidden layer containing a finite number of neurons can approximate any continuous function on compact/bounded subsets of $\mathbb{R}^n$ to arbitrary accuracy $\varepsilon > 0$.

Key Points

- Only ONE hidden layer needed
- Requires non-linear activation
- May need many neurons
- Doesn't guarantee efficient learning

Practical Significance

- Theoretical foundation for neural networks
- Proves MLPs can solve complex problems
- Deep networks often more efficient in practice
- Justifies using multiple layers

Forward Propagation

Computing Network Output



What is Forward Propagation?

Process of computing output from input by passing through layers

Information flows forward: Input → Hidden → Output

Each layer transforms data using weights and activation functions

Step 1: Linear Combination

$$\mathbf{z} = \mathbf{W} \cdot \mathbf{a} + \mathbf{b}$$

Weighted sum of inputs

Step 2: Activation

$$\mathbf{a} = \sigma(\mathbf{z})$$

Apply non-linear function

Forward Propagation: Mathematics

Layer-by-Layer Computation

For each layer $l = 1, 2, \dots, L$:

$$z^{(l)} = W^{(l)} \cdot a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = \sigma(z^{(l)})$$

Where:

- $z^{(l)}$ = pre-activation (weighted sum) at layer l
- $a^{(l)}$ = activation output at layer l ($a^{(0)}$ = input x)
- $W^{(l)}$ = weight matrix connecting layer $l-1$ to layer l
- $b^{(l)}$ = bias vector for layer l

Forward Propagation: Example

3-Layer Network (Input → Hidden → Output)

Layer 1: Input → Hidden

$$z^{(1)} = W^{(1)} \cdot x + b^{(1)} \quad a^{(1)} = \text{ReLU}(z^{(1)})$$

Layer 2: Hidden → Output

$$z^{(2)} = W^{(2)} \cdot a^{(1)} + b^{(2)} \quad \hat{y} = \text{Softmax}(z^{(2)})$$

Algorithm Steps:

1. Input: x → 2. Compute $z^{(1)}, a^{(1)}$ → 3. Compute $z^{(2)}, \hat{y}$ → 4. Output: $\hat{y}$

Backpropagation Algorithm

Training Multi-layer Networks



What is Backpropagation?

Algorithm for computing gradients of the loss with respect to all weights

Propagates error backwards through the network

Efficiently applies the chain rule of calculus

Foundation of modern neural network training

Why Do We Need It?

- Cannot directly compute gradients for hidden layers
- Naive gradient computation would be extremely slow
- Backpropagation reuses computations efficiently

Backpropagation: Intuition

The Chain Rule

If $y = f(u)$ and $u = g(x)$, then:

$$dy/dx = (dy/du) \times (du/dx)$$

Error Flow

Forward: Input $\rightarrow$ Hidden $\rightarrow$ Output $\rightarrow$ Loss

Backward: Loss $\leftarrow$ Hidden $\leftarrow$ Input (propagate gradients)

Backpropagation: Output Layer

Step 1: Computing Gradients at Output

Loss Function (e.g., Mean Squared Error)

$$L = (1/2) \sum (y - \hat{y})^2$$

Gradient Computation for Output Layer L:

1. Error term (delta):

$$\delta^{(L)} = \partial L / \partial z^{(L)} = (a^{(L)} - y) \odot \sigma'(z^{(L)})$$

2. Weight gradient:

$$\partial L / \partial W^{(L)} = \delta^{(L)} \cdot (a^{(L-1)})^T$$

3. Bias gradient:

$$\partial L / \partial b^{(L)} = \delta^{(L)}$$

Backpropagation: Hidden Layers

Step 2: Propagating Error Backwards

For each hidden layer $l = L-1, L-2, \dots, 1$:

1. Propagate error from layer $l+1$ to layer l :

$$\delta^{(l)} = (W^{(l+1)})^T \cdot \delta^{(l+1)} \odot \sigma'(z^{(l)})$$

2. Compute gradients for layer l :

Weight gradient:

$$\partial L / \partial W^{(l)} = \delta^{(l)} \cdot (a^{(l-1)})^T$$

Bias gradient:

$$\partial L / \partial b^{(l)} = \delta^{(l)}$$

$\odot$ denotes element-wise (Hadamard) product

Complete Backpropagation Algorithm

1

Forward Pass

Compute all activations $a^{(l)}$ and $z^{(l)}$

2

Compute Loss

Calculate $L = \text{loss}(\hat{y}, y)$

3

Output Layer δ

$\delta^{(L)} = (a^{(L)} - y) \odot \sigma'(z^{(L)})$

4

Backward Pass

$\delta^{(l)} = ((W^{(l+1)})^T \cdot \delta^{(l+1)}) \odot \sigma'(z^{(l)})$

5

Compute Gradients

$\partial L / \partial W^{(l)}$ and $\partial L / \partial b^{(l)}$ for all layers

6

Update Weights

$W^{(l)} \leftarrow W^{(l)} - \eta \cdot \partial L / \partial W^{(l)}$

Backpropagation: Key Insights



Advantages

- Efficient: $O(n)$ complexity
- Exact gradients
- Enables deep networks
- Foundation of modern ML

Important Notes

- Relies on chain rule
- Requires differentiable activation
- Can suffer from vanishing/exploding gradients
- Stores intermediate values

Computational Efficiency

Without backprop: Would need to compute gradient for each weight independently $\rightarrow O(n^2)$ or worse

With backprop: Single forward + backward pass $\rightarrow O(n)$ where n = number of weights

Computational Graph View



Understanding Flow of Computation

Forward Pass: Compute values

$$\mathbf{x} \rightarrow \mathbf{z}^{(1)} \rightarrow \mathbf{a}^{(1)} \rightarrow \mathbf{z}^{(2)} \rightarrow \mathbf{a}^{(2)} \rightarrow \dots \rightarrow \mathbf{z}^{(\square)} \rightarrow \hat{\mathbf{y}} \rightarrow \mathcal{L}$$

Each node computes a value based on its inputs

Backward Pass: Compute gradients

$$\partial \mathcal{L} / \partial \mathbf{W}^{(\square)} \leftarrow \partial \mathcal{L} / \partial \mathbf{a}^{(\square)} \leftarrow \dots \leftarrow \partial \mathcal{L} / \partial \mathbf{a}^{(2)} \leftarrow \partial \mathcal{L} / \partial \mathbf{a}^{(1)} \leftarrow \partial \mathcal{L} / \partial \mathbf{x}$$

Each node passes gradients to its predecessors using chain rule

Summary

- MLPs can approximate any continuous function (Universal Approximation)
- Forward propagation computes output layer by layer
- Backpropagation efficiently computes gradients using chain rule
- Error propagates backwards to update all weights
- Foundation enables training of deep neural networks